

AGH WEAiIB	Mikroelektronika w Technice i Medycynie	Magdalena Tatko 416630 Agnieszka Wróblewska 414619	Systemy dedykowane w Układach Programowalnych	Data wykonania 11.05.2026
---------------	--	---	---	------------------------------

Max/Min Reduction

REPOZYTORIUM PROJEKTU

https://github.com/mtatko03/sdup_projekt_MT_AW.git

1. Wybór algorytmu

Do realizacji projektu wybrano algorytm **Max/Min Reduction**, służący do wyznaczania najmniejszej i największej wartości w zbiorze danych. Jest to operacja redukcji, ponieważ wiele danych wejściowych zostaje sprowadzonych do dwóch wyników końcowych: minimum i maksimum.

Algorytm wybrano ze względu na prostotę oraz łatwość implementacji w językach Python i Verilog. Wykorzystuje on jedynie operacje porównania, dzięki czemu dobrze nadaje się do realizacji w układzie FPGA. Projekt umożliwia również zapoznanie się z komparatorami, drzewem redukcyjnym, potokowaniem oraz interfejsami AXI-Lite i AXI-Stream.

- **Założenia projektowe**

W pierwszym etapie przyjęto, że dane wejściowe są zapisane w pamięci w postaci jednowymiarowej tablicy próbek. Model behawioralny w Pythonie generuje przykładową zawartość pamięci, a następnie wyszukuje w niej wartość minimalną oraz maksymalną.

Dla głównej wersji projektu przyjęto pamięć zawierającą **1024 wartości**. Dane są generowane jako liczby **16-bitowe ze znakiem**, co oznacza, że pełny dopuszczalny zakres wartości wynosi od **-32768 do 32767**. Taki wybór jest zgodny z późniejszą implementacją w języku Verilog, gdzie konieczne będzie jawne określenie szerokości danych oraz typu signed albo unsigned.

W celu uzyskania bardziej zróżnicowanych wyników testów wprowadzono możliwość losowania węższego zakresu wartości wewnątrz pełnego zakresu 16-bitowego. Dzięki temu wyniki minimum i maksimum nie są przy każdym uruchomieniu prawie zawsze bliskie wartościom skrajnym całego zakresu, lecz zależą od aktualnie wygenerowanej zawartości pamięci.

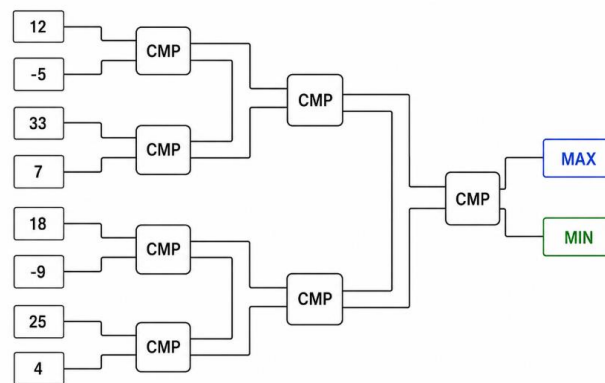
Założono również, że algorytm powinien działać nie tylko dla liczby próbek będącej potęgą dwójki, takiej jak 1024, ale również dla nieparzystej liczby elementów. W takim przypadku ostatni element na danym poziomie drzewa, który nie ma pary do porównania, jest przepisywany bez zmian na kolejny poziom redukcji.

2. Opis algorytmu

Algorytm Max/Min Reduction został zrealizowany z wykorzystaniem **drzewa porównań**, czyli struktury typu reduction tree. Oznacza to, że dane nie są analizowane wyłącznie jako jedna długa sekwencja, lecz są dzielone na pary i redukowane poziomami. Na początku wszystkie dane wejściowe traktowane są jako poziom zerowy drzewa. Następnie wartości są porównywane parami. Z każdej pary wybierana jest mniejsza wartość dla drzewa minimum oraz większa wartość dla drzewa maksimum. Otrzymane wyniki tworzą kolejny poziom drzewa, na którym ponownie wykonywane są porównania parami. Proces powtarza się aż do momentu, w którym pozostanie jedna wartość minimalna oraz jedna wartość maksymalna. Dla 1024 próbek liczba poziomów redukcji wynosi 10, ponieważ:

$$\log_2(1024) = 10$$

Oznacza to, że liczba elementów zmniejsza się w kolejnych poziomach następująco: $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$



W implementacji programowej w Pythonie kolejne poziomy są wyznaczane za pomocą pętli. Natomiast w przyszłej implementacji sprzętowej w FPGA porównania na jednym poziomie drzewa mogą być wykonywane równolegle, co daje możliwość znaczącej akceleracji algorytmu.

3. Funkcja w Pythonie i praktyczne zastosowanie

Algorytm wyznaczania wartości minimalnej i maksymalnej ma wiele praktycznych zastosowań w systemach cyfrowego przetwarzania danych. Przykładowe zastosowania:

- analiza sygnałów z przetworników ADC,
- wykrywanie zakresu amplitudy sygnału,
- przetwarzanie obrazów,
- analiza danych pomiarowych,
- monitorowanie wartości granicznych w systemach wbudowanych,
- detekcja anomalii w strumieniu danych,
- przygotowanie danych do dalszego przetwarzania, na przykład normalizacji.

W układach FPGA taki algorytm może działać w czasie rzeczywistym, analizując dane zapisane w pamięci lub napływające z innych modułów cyfrowych. Dzięki wykorzystaniu drzewa porównań możliwe jest również zrównoleglenie operacji, ponieważ wiele porównań może zostać wykonanych jednocześnie.

Na potrzeby pierwszego etapu projektu przygotowano model behawioralny w języku Python. Model składa się z trzech głównych funkcji:

Funkcja	Opis działania	Znaczenie w projekcie
<code>generate_memory()</code>	Generuje tablicę losowych wartości 16-bitowych ze znakiem w zadanym zakresie.	Tworzy przykładowe dane wejściowe reprezentujące zawartość pamięci wejściowej.
<code>reduction_tree_min_max()</code>	Realizuje algorytm drzewa porównań, który wyznacza minimalną i maksymalną wartość w tablicy danych.	Stanowi właściwy algorytm projektowy (Min/Max Reduction).
<code>verify_result()</code>	Porównuje wyniki algorytmu (minimum i maksimum) z funkcjami wzorcowymi <code>min()</code> oraz <code>max()</code> dostępnymi w Pythonie.	Sprawdza poprawność działania modelu i weryfikuje uzyskane wyniki.

- KOD PROGRAMU:

```

1  import random
2
3
4  def generate_memory(size=1024, data_width=16, signed=True, random_range=True):
5
6      if signed:
7          absolute_min = -(2 ** (data_width - 1))
8          absolute_max = (2 ** (data_width - 1)) - 1
9      else:
10         absolute_min = 0
11         absolute_max = (2 ** data_width) - 1
12
13         if random_range:
14             center = random.randint(absolute_min // 2, absolute_max // 2)
15             spread = random.randint(100, 5000)
16
17             min_value = max(absolute_min, center - spread)
18             max_value = min(absolute_max, center + spread)
19         else:
20             min_value = absolute_min
21             max_value = absolute_max
22
23         memory = []
24
25         for _ in range(size):
26             value = random.randint(min_value, max_value)
27             memory.append(value)
28
29         print(f"Generated memory: {size} values")
30         print(f"Data width: {data_width} bits")
31         print(f"Signed: {signed}")
32         print(f"Absolute value range: {absolute_min} to {absolute_max}")
33         print(f"Generated value range: {min_value} to {max_value}\n")
34
35         return memory
36
37
38  def reduction_tree_min_max(data):
39
40      if len(data) == 0:
41          raise ValueError("Memory is empty. Cannot find MIN and MAX.")
42
43      min_level = data.copy()
44      max_level = data.copy()
45
46      level = 0
47
48      print("START REDUCTION TREE")
49      print(f"Level {level}: number of elements = {len(data)}")

```

```

51     while len(min_level) > 1:
52         next_min_level = []
53         next_max_level = []
54
55         for i in range(0, len(min_level), 2):
56             if i + 1 >= len(min_level):
57                 next_min_level.append(min_level[i])
58                 next_max_level.append(max_level[i])
59                 continue
60
61                 a_min = min_level[i]
62                 b_min = min_level[i + 1]
63
64                 a_max = max_level[i]
65                 b_max = max_level[i + 1]
66
67                 if a_min < b_min:
68                     local_min = a_min
69                 else:
70                     local_min = b_min
71
72                 if a_max > b_max:
73                     local_max = a_max
74                 else:
75                     local_max = b_max
76
77                 next_min_level.append(local_min)
78                 next_max_level.append(local_max)
79
80     min_level = next_min_level
81     max_level = next_max_level
82
83     level += 1
84     print(f"Level {level}: number of elements = {len(min_level)}")
85
86     final_min = min_level[0]
87     final_max = max_level[0]
88
89     return final_min, final_max
90
91
92     def verify_result(memory, result_min, result_max):
93         reference_min = min(memory)
94         reference_max = max(memory)
95
96         print("\nRESULT")
97         print("MIN =", result_min)
98         print("MAX =", result_max)
99
100        print("\nREFERENCE")
101        print("Python min =", reference_min)
102        print("Python max =", reference_max)
103
104        if result_min == reference_min and result_max == reference_max:
105            print("\nVerification: PASSED")
106        else:
107            print("\nVerification: FAILED")
108
109    memory = generate_memory(size=1024, data_width=16, signed=True, random_range=True)
110
111    result_min, result_max = reduction_tree_min_max(memory)
112
113    verify_result(memory, result_min, result_max)

```

- **Wynik działania modelu**

Po uruchomieniu programu generowana jest pamięć zawierająca 1024 losowe wartości 16-bitowe ze znakiem. Program wyświetla podstawowe informacje o danych wejściowych, takie jak rozmiar pamięci, szerokość danych oraz zakres wygenerowanych wartości. Następnie wykonywany jest algorytm drzewa porównań, którego wynikiem są wartości minimalna i maksymalna. Otrzymane wyniki są porównywane z wartościami wyznaczonymi za pomocą

wbudowanych funkcji Pythona *min()* oraz *max()*. Pojawienie się komunikatu *Verification: PASSED* oznacza, że algorytm poprawnie wyznaczył minimum i maksimum dla wygenerowanego zestawu danych.

```
PS C:\Users\aga20\Desktop\SDUP> python project_SDUP_V5.py
Generated memory: 1024 values
Data width: 16 bits
Signed: True
Absolute value range: -32768 to 32767
Generated value range: 12547 to 13489

START REDUCTION TREE
Level 0: number of elements = 1024
Level 1: number of elements = 512
Level 2: number of elements = 256
Level 3: number of elements = 128
Level 4: number of elements = 64
Level 5: number of elements = 32
Level 6: number of elements = 16
Level 7: number of elements = 8
Level 8: number of elements = 4
Level 9: number of elements = 2
Level 10: number of elements = 1

RESULT
MIN = 12549
MAX = 13489

REFERENCE
Python min = 12549
Python max = 13489

Verification: PASSED
```

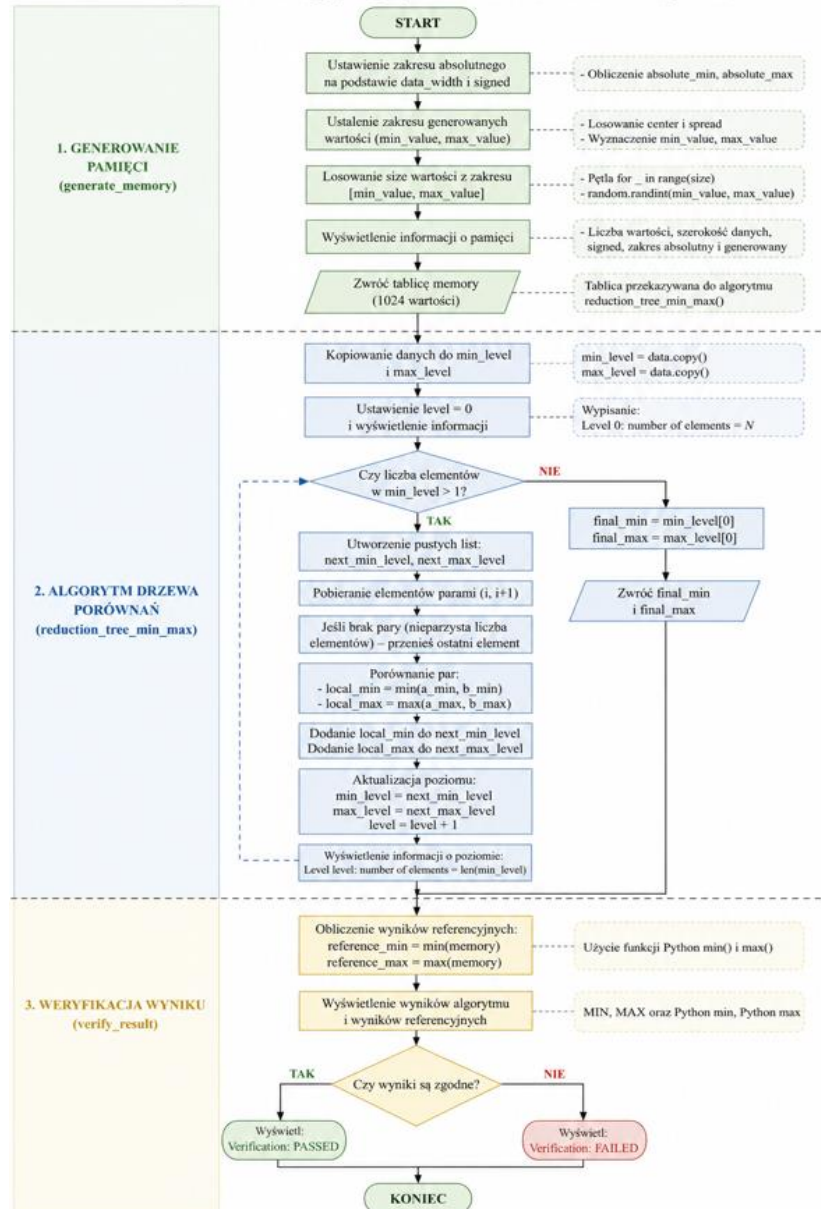
4. Algorytm w postaci diagramu sekwencyjnego

Diagram sekwencyjny przedstawia sposób działania algorytmu Max/Min Reduction w formie zbliżonej do pracy modułu sprzętowego. Po wykonaniu resetu moduł oczekuje na sygnał *start*, który rozpoczyna przetwarzanie danych. Następnie inicjalizowane są podstawowe sygnały sterujące oraz licznik próbek *index*.

W każdym cyklu działania modułu odczytywana jest jedna próbka danych *sample = data[index]*. Pierwsza próbka służy do inicjalizacji rejestrów *min_reg* oraz *max_reg*. Dla kolejnych próbek wykonywane są dwa porównania: sprawdzane jest, czy aktualna próbka jest mniejsza od wartości zapisanej w *min_reg* oraz czy jest większa od wartości zapisanej w *max_reg*. W przypadku spełnienia warunku odpowiedni rejestr zostaje zaktualizowany.

Po przetworzeniu próbki licznik *index* jest zwiększany o jeden. Proces powtarza się do momentu przetworzenia wszystkich elementów zbioru danych. Po zakończeniu pracy ustawiany jest sygnał *done = 1*, a na wyjściu dostępne są końcowe wartości minimum i maksimum zapisane odpowiednio w *min_reg* i *max_reg*.

Diagram sekwencyjny algorytmu Max/Min Reduction (Python)



• Złożoność algorytmu

W drzewie porównań liczba elementów zmniejsza się o połowę na każdym poziomie. Dla $N = 1024$ próbek liczba poziomów redukcji wynosi: $\log_2(1024) = 10$. W modelu programowym obliczenia wykonywane są sekwencyjnie, dlatego złożoność czasowa wynosi $O(N)$. Głębokość drzewa wynosi natomiast $O(\log N)$. W przyszłej implementacji FPGA porównania na jednym poziomie mogą być realizowane równolegle, co umożliwia przyspieszenie działania algorytmu.

• Ograniczenia obecnego modelu

Obecny model jest modelem behawioralnym i nie zawiera jeszcze rzeczywistego interfejsu sprzętowego. Dane wejściowe są generowane w Pythonie i przechowywane w liście, która pełni rolę uproszczonego modelu pamięci. Model nie uwzględnia jeszcze rzeczywistego dostępu do pamięci FPGA, magistrali AXI-Lite ani AXI-Stream. Nie zawiera również sygnałów takich jak

data_valid, *data_ready*, *tlast*, *start* czy *done*. Na tym etapie nie jest to jednak problemem, ponieważ celem było przygotowanie i sprawdzenie samego algorytmu wyszukiwania wartości minimalnej i maksymalnej. W kolejnych etapach projekt zostanie zamknięty w moduł sprzętowy. Konieczne będzie określenie wejść i wyjść, sposobu dostępu do danych oraz przygotowanie opisu w języku Verilog. Następnie zostanie przygotowany testbench, który umożliwi potwierdzenie poprawności działania modułu syntezywalnego.

5. Opis behawioralny

- Opis działania modułu

W kolejnym etapie projektu przygotowano opis behawioralny modułu w języku **SystemVerilog**. Moduł otrzymał nazwę *max_min_reduction_beh* i realizuje wyznaczanie wartości minimalnej oraz maksymalnej spośród 1024 kolejno podawanych próbek wejściowych. Dane wejściowe mają szerokość 16 bitów i są interpretowane jako liczby ze znakiem. Oznacza to, że moduł może poprawnie przetwarzać zarówno wartości dodatnie, jak i ujemne z zakresu od -32768 do 32767. Liczba próbek oraz szerokość danych zostały zapisane jako parametry modułu:

parameter int DATA_WIDTH = 16 i parameter int N = 1024

Dzięki temu moduł jest parametryzowany i w razie potrzeby można zmienić liczbę analizowanych próbek lub szerokość danych bez przebudowy całej struktury programu. Działanie modułu rozpoczyna się po ustawieniu sygnału *start*. Następnie moduł przechodzi do stanu ładowania danych. Każda próbka wejściowa jest przekazywana przez port *data_in*, a sygnał *data_valid* informuje, że dana znajdująca się na wejściu jest poprawna i może zostać przyjęta przez moduł. Pierwsza odebrana próbka inicjalizuje jednocześnie wartość minimalną oraz maksymalną. Kolejne próbki są porównywane z aktualnymi wartościami *min_out* oraz *max_out*. Jeżeli nowa próbka jest mniejsza od aktualnego minimum, następuje aktualizacja *min_out*. Jeżeli jest większa od aktualnego maksimum, aktualizowane jest *max_out*. Po przyjęciu wszystkich 1024 próbek moduł ustawia sygnał *done = 1*, co oznacza zakończenie pracy i dostępność poprawnych wyników na wyjściach *min_out* oraz *max_out*.

6. Zamknięcie w moduł, określenie I/O i sposobu dostępu do danych

Algorytm został zamknięty w osobny moduł SystemVerilog o nazwie *max_min_reduction_beh*. Oznacza to, że logika wyszukiwania wartości minimalnej i maksymalnej została opisana jako blok sprzętowy z jednoznacznie określonymi wejściami, wyjściami oraz sygnałami sterującymi. Interfejs modułu został dobrany tak, aby odpowiadał późniejszej możliwości podłączenia modułu do prostego systemu sterującego lub do interfejsu AXI-Lite. Dane wejściowe nie są przekazywane jako jedna duża tablica, lecz jako kolejne próbki podawane na wejście *data_in*. Takie rozwiązanie jest bardziej zbliżone do

rzeczywistego działania układu cyfrowego, w którym dane mogą napływać sekwencyjnie z pamięci, procesora lub innego modułu.

Nazwa portu	Kierunek	Szerokość	Opis
clk	Wejście	1 bit	Zegar systemowy
rst_n	Wejście	1 bit	Reset (aktywny niskim stanem)
start	Wejście	1 bit	Sygnal rozpoczęcia obliczeń
data_in	Wejście	16 bitów	Szyna danych wejściowych
data_valid	Wejście	1 bit	Informacja, że na szynie jest poprawna dana
min_out	Wyjście	16 bitów	Wyliczona wartość minimalna
max_out	Wyjście	16 bitów	Wyliczona wartość maksymalna
done	Wyjście	1 bit	Sygnal zakończenia pracy modułu

W module zastosowano również wewnętrzną pamięć:

logic signed [DATA_WIDTH-1:0] memory [0:N-1];

Pamięć ta służy do zapisu przyjmowanych próbek. Jednocześnie podczas ładowania danych moduł aktualizuje wartości *min_out* oraz *max_out*, dzięki czemu nie trzeba wykonywać dodatkowego przeglądania pamięci po zakończeniu zapisu wszystkich próbek. Liczba odebranych próbek jest kontrolowana przez licznik *sample_count*. Jego szerokość została wyznaczona automatycznie za pomocą:

localparam int COUNT_WIDTH = \$clog2(N + 1);

Dla $N = 1024$ licznik musi umożliwiać zliczenie wszystkich próbek od 0 do 1023.

7. Potwierdzenie prawidłowego działania modułu

Do sprawdzenia poprawności działania modułu przygotowano testbench *tb_max_min_reduction_beh.sv*. Testbench pełni rolę środowiska symulacyjnego, które generuje dane wejściowe, podaje je do modułu oraz sprawdza poprawność uzyskanych wyników. W testbenchu utworzono tablicę:

logic signed [DATA_WIDTH-1:0] test_memory [0:N-1];

Tablica *test_memory* zawiera 1024 losowo wygenerowane próbki 16-bitowe ze znakiem. Dane są generowane za pomocą funkcji *\$urandom_range(0, 65535)*, a następnie interpretowane jako wartości typu *signed*. Dzięki temu w testach pojawiają się zarówno wartości dodatnie, jak i ujemne. Na podstawie zawartości *test_memory* testbench oblicza wartości referencyjne *reference_min* oraz *reference_max*. Są one wyznaczone niezależnie od testowanego modułu, aby później można było porównać je z wynikami *min_out* i *max_out*. Po przygotowaniu danych testbench ustawia sygnał *start = 1*, a następnie podaje kolejne próbki na wejście *data_in*. Dane są podawane na zboczu opadającym zegara, aby były stabilne przed kolejnym zboczem narastającym, na którym moduł je odczytuje. W trakcie podawania danych

aktywny jest sygnał *data_valid*. Po przyjęciu wszystkich próbek testbench oczekuje na ustawienie sygnału:

done = 1

Następnie wypisywane są wyniki działania modułu oraz wartości referencyjne. Jeżeli wartości *min_out* i *max_out* są zgodne z *reference_min* i *reference_max*, *testbench* wyświetla komunikat:

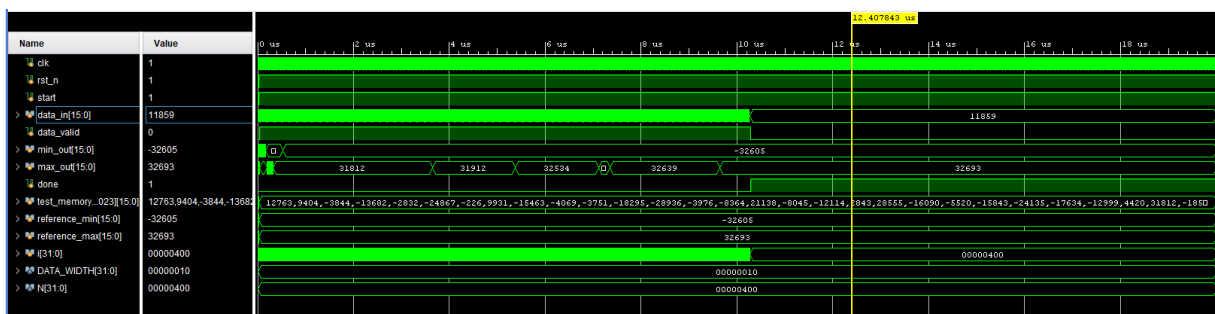
Verification: PASSED

W przeprowadzonej symulacji otrzymano:

```
=====
RESULT FROM MODULE:
MIN = -32605
MAX = 32693
REFERENCE RESULT:
MIN = -32605
MAX = 32693
Verification: PASSED
DONE = 1
) =====
```

Oznacza to, że moduł poprawnie wyznaczył wartość minimalną oraz maksymalną dla zestawu 1024 próbek wejściowych. Dodatkowo ustawienie *DONE = 1* potwierdza, że moduł prawidłowo zakończył pracę po przetworzeniu wszystkich danych.

Na przebiegach czasowych można zauważyć, że podczas podawania próbek zmienia się sygnał *data_in*, aktywny jest sygnał *data_valid*, a wartości *min_out* oraz *max_out* są aktualizowane w trakcie działania modułu. Po zakończeniu przetwarzania sygnał *done* przechodzi w stan wysoki.



8. Opis strukturalny / syntezywalny

Przygotowany moduł *max_min_reduction_beh* ma strukturę możliwą do interpretacji jako opis RTL. W module zastosowano zegar, reset, rejestry, licznik próbek, pamięć wewnętrzną oraz prostą maszynę stanów. Dzięki temu projekt nie jest jedynie abstrakcyjnym opisem

algorytmu, ale stanowi opis bloku cyfrowego możliwego do dalszej syntezy w środowisku Vivado.

Główne elementy struktury modułu to:

- maszyna stanów FSM,
- licznik próbek *sample_count*,
- pamięć wewnętrzna *memory*,
- rejestr wyjściowy minimum *min_out*,
- rejestr wyjściowy maksimum *max_out*,
- sygnał zakończenia pracy *done*.

Maszyna stanów składa się z trzech stanów:

Stan	Opis
IDLE	stan oczekiwania na sygnał start
LOAD	stan ładowania próbek i aktualizacji minimum oraz maksimum
DONE_STATE	stan zakończenia pracy, w którym <i>done</i> = 1

W stanie *IDLE* moduł zeruje licznik próbek, sygnał *done* oraz rejestry wynikowe. Po pojawieniu się sygnału start moduł przechodzi do stanu *LOAD*. W stanie *LOAD* każda poprawna próbka oznaczona sygnałem *data_valid* jest zapisywana do pamięci wewnętrznej. Pierwsza próbka inicjalizuje rejestry *min_out* i *max_out*, natomiast kolejne próbki są porównywane z aktualnymi wartościami tych rejestrów. W zależności od wyniku porównania następuje aktualizacja minimum lub maksimum. Po przyjęciu ostatniej próbki, czyli gdy:

$$sample_count == N - 1$$

moduł ustawia *done* = 1 i przechodzi do stanu *DONE_STATE*. W stanie *DONE_STATE* wynik pozostaje dostępny na wyjściach *min_out* i *max_out*, a sygnał *done* utrzymuje wartość 1. Dopiero zdjęcie sygnału *start* powoduje powrót modułu do stanu *IDLE*.

Opis można uznać za synteżowalny na poziomie podstawowym, ponieważ wykorzystuje typowe elementy projektowania cyfrowego: synchroniczny blok *always_ff*, rejestry, licznik oraz instrukcję *case* opisującą automat stanów. Na tym etapie projektu nie zastosowano jeszcze interfejsu AXI-Lite ani AXI-Stream. Te elementy należą do dalszych etapów rozbudowy projektu, związanych z uzbrojeniem modułu i podłączeniem go do procesora.

WNIOSKI

W ramach projektu przygotowano model algorytmu Max/Min Reduction służącego do wyznaczania wartości minimalnej i maksymalnej w zbiorze danych. W pierwszym etapie wykonano model behawioralny w języku Python, który umożliwił sprawdzenie poprawności działania algorytmu na pamięci zawierającej 1024 losowo wygenerowane wartości 16-bitowe ze znakiem. Wyniki uzyskane przez algorytm zostały porównane z funkcjami referencyjnymi *min()* oraz *max()*, co potwierdziło poprawność działania modelu.

Następnie algorytm został przeniesiony do języka SystemVerilog i zamknięty w osobny moduł *max_min_reduction_beh*. Moduł posiada określone wejścia, wyjścia oraz sygnały sterujące, takie jak *start*, *data_valid* i *done*. Dane wejściowe są podawane sekwencyjnie przez port *data_in*, a moduł na bieżąco aktualizuje wartości *min_out* oraz *max_out*. Po przetworzeniu wszystkich próbek ustawiany jest sygnał *done*, który informuje o zakończeniu pracy.

Poprawność działania modułu została potwierdzona w testbenchu. Testbench generuje dane wejściowe, oblicza wartości referencyjne, podaje próbki do modułu oraz porównuje wyniki uzyskane z modułu z wartościami referencyjnymi. Otrzymany komunikat *Verification: PASSED* oraz ustawienie *DONE = 1* potwierdzają, że moduł poprawnie wyznacza minimum i maksimum dla zestawu danych wejściowych.

Projekt pozwolił na przejście od programowego modelu algorytmu do opisu sprzętowego w języku SystemVerilog. Pokazano, że nawet prosty algorytm redukcji wymaga w implementacji sprzętowej określenia sposobu dostępu do danych, sygnałów sterujących, liczników, rejestrów oraz warunków zakończenia pracy. Obecna wersja realizuje podstawowy, sekwencyjny sposób podawania danych, natomiast w kolejnych etapach projekt mógłby zostać rozbudowany o interfejs AXI-Lite lub AXI-Stream oraz o bardziej równoległą strukturę drzewa porównań.